



University of Piraeus
School of Information and Communication Technologies
Department of Digital Systems

Level: Master's Programme of Studies

Network Security

Case 7

Supervising Professor: Prof. Christos Xenakis

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

24/12/2025

Abstract

Implementation of a «Defense-in-Depth» security architecture on an Ubuntu Server, utilizing iptables (Network Firewall) for port filtering and ModSecurity (WAF) for web application protection. Enforcement of a strict security policy permitting only SSH, HTTP/S, and port 9001 traffic, while systematically dropping all other incoming packets. Verification of system integrity via penetration testing from Kali Linux, demonstrating the effective blocking of unauthorized access by the firewall and the prevention of SQL Injection attacks by the WAF.

Contents

Introduction.....	
Practical part of the assignment.....	
Bibliography.....	

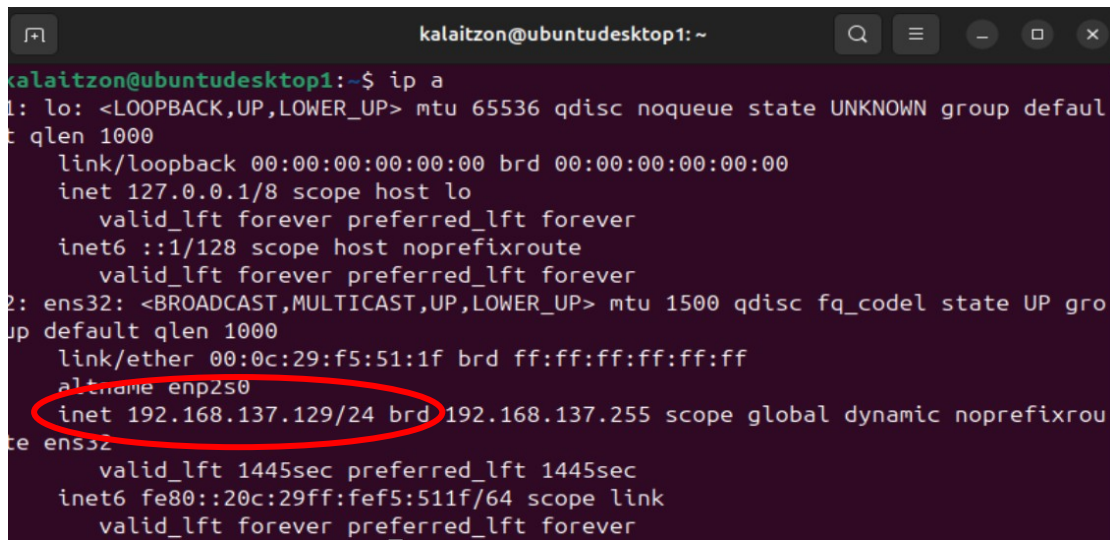
Introduction

Within this lab exercise, the implementation and configuration of a security architecture based on the 'Defense-in-Depth' model was carried out. For the needs of the scenario, two Virtual Machines with distinct roles were used, simulating an attack and defence environment.

The first system (Ubuntu) operates in a hybrid manner as a Server and a security node, applying Host-based Security measures. As a Server, it hosts the necessary services, specifically the Apache Web Server for handling HTTP/HTTPS traffic, as well as a custom service on port 9001. Unlike infrastructures based exclusively on perimeter firewalls, this implementation enables security directly on the Server through two layers of control: on the one hand iptables (Network Firewall) for filtering network packets, and on the other ModSecurity (Web Application Firewall — WAF) for detecting malicious data within the applications.

The second system (Kali Linux) takes on the role of the Client/Attacker. Through it, connectivity checks and attack simulations are carried out, with the aim of verifying the integrity and effectiveness of the filtering rules applied on the Ubuntu VM.

Practical part of the assignment



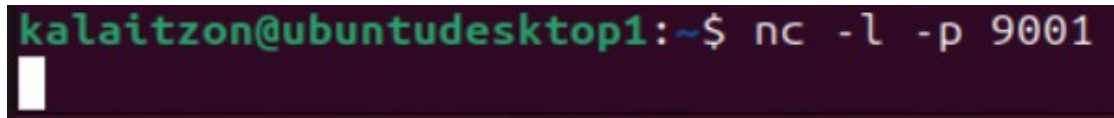
```
kalaitzon@ubuntudesktop1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixroute ens32
        valid_lft 1445sec preferred_lft 1445sec
    inet6 fe80::20c:29ff:fef5:511f/64 scope link
        valid_lft forever preferred_lft forever
```

First, we must find the Ubuntu IP with the command `ip a`. The IP is: 192.168.137.129

Next, we will download and update the following applications:

- **Apache2:** It is the most popular web server and is required for HTTP/HTTPS to work. It is the server with which we will create the website that the client (Kali Linux) will connect to.
- **Libapache2-mod-security2:** It is the web application firewall (WAF). It provides security to the server against web attacks. Essentially, it sits within Apache and inspects the data to block attacks such as SQL injections, etc.
- **Netcat-openbsd:** It is the tool we will use to read and write data over network connections. Because the assignment brief has, as a basic requirement, that port 9001 be open, we will use netcat to open this port. In the screenshot we observe that this command, which installs and updates the above applications, does not work because we did not have administrative privileges. By adding 'sudo' in front of the rest of the command, the problem is solved.

Next, we enable the web server with the command 'sudo service apache2 start', and to check whether the apache server works, we use the command 'sudo service apache2 status'. The result, as shown in the screenshot, shows that everything went well and that the server is open and waiting for HTTP/HTTPS connections.



```
kalaitzon@ubuntudesktop1:~$ nc -l -p 9001
```

Here we will use the netcat tool, and with the command `nc -l -p 9001` we will create a network service. With the '-l' parameter we tell the system to wait for a connection (listen) instead of connecting somewhere else. With the '-p' parameter we specify which port to open, i.e. 9001. Without this command, even if we configured the firewall to allow 9001, the port would be closed because there would be no program using it. As soon as we run the command, the cursor will blink, which means the command is running normally and the server is in a waiting state. We will leave this terminal open so that the port continues to 'listen', and we will open a new terminal.

Next, we will create the firewall rules using the iptables tool. Essentially, with the firewall we will determine who is allowed in and who is not.

With the command 'sudo iptables -F' we delete (-F) all the previous rules. We do this in order to have a 'clean' base, so that no problem is created by the pre-existing ones.

With the command 'sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT', we add a rule for the TCP protocol on port 22, with an instruction to accept it (ACCEPT). We open the port for SSH so that remote connection to the server is possible.

With the command 'sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT' and the command 'sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT' we allow traffic on ports 80 and 443. Essentially, we open access for the web server (HTTP/HTTPS).

With the command 'sudo iptables -A INPUT -p tcp --dport 9001 -j ACCEPT' we allow traffic on port 9001. We did this because it is a specific requirement of Use Case 7, in order to allow access to the custom service that the server hosts to serve the client.

With the command 'sudo iptables -A INPUT -i lo -j ACCEPT' we allow traffic on the loopback interface (lo). In other words, we allow the server to talk to itself. We do this because many system services communicate with each other via the localhost (127.0.0.1), and blocking this traffic would cause malfunctions in the operating system.

With the command 'sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT' we allow packets belonging to

already open connections. Essentially, we ensure that if the server requests something from the internet, the response will be able to return. Without this, the firewall would cut off the responses to its own requests.

With the command 'sudo iptables -P INPUT DROP' and the command 'sudo iptables -P FORWARD DROP' we change the policy (-P) to DROP. This is the most important part among the commands we wrote, because if a packet does not match any of the above rules (22, 80, 443, 9001), then it is ignored. That is, we implement here a state in which all ports are closed except for the ones mentioned.

Finally, with the command 'sudo iptables -L -v -n' we displayed the list of active rules for confirmation. The list confirms what we did above (the ports we want are ACCEPT and the general policy is DROP).

Next, with the command 'sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf' we copy the file with the recommended settings and create a main configuration file (modsecurity.conf). We do this because modsecurity does not have an active configuration file. We must create this file for it to start working correctly. Then, with the command 'sudo nano /etc/modsecurity/modsecurity.conf' we open this file and change the setting from 'SecRuleEngine DetectionOnly' to 'SecRuleEngine On', as shown in the image below.



This was the last step for ModSecurity to block the attacks. Finally, with the command 'sudo service apache2 restart' we restart the web server so that all the changes we made in the configuration file are applied and read by the system, in order to enable the WAF.

Now we will move to the client (Kali Linux) and use the Ubuntu (server) IP in order to check port 80. We will do this by typing the IP into the client's browser.

Since it shows us this page, it means that everything works as it should. Next, we will check port 9001 with the Netcat tool. We will open a terminal on Kali Linux and type the command 'nc 192.168.137.129 9001'. We observe that the connection is successful, since the cursor is blinking.

To confirm it, we will type the message 'HELLO' right below and press Enter. We will temporarily return to Ubuntu and go to the terminal we had left open.

We observe that the message 'HELLO' appeared here as well. This is proof that the communication works. In other words, the client managed to connect successfully to port 9001 of Ubuntu (server). So the iptables rule we created earlier works correctly. If the firewall were misconfigured, this message would never arrive and would not appear in the server's terminal. Finally, with the command 'nmap -p 8080 192.168.137.129' we check port 8080 on Ubuntu from Kali Linux with the Nmap tool. The result, as we observe, is that the firewall dropped (DROP) it and sent no response back. So iptables on Ubuntu did its job correctly.

For confirmation, we added to the Ubuntu URL the parameter '?id=1 AND 1=1' as a simulation of an SQL Injection attack on the web server. As we observe, the web server returned a '403 Forbidden' error message. Instead of showing the normal page, the system refused to serve the request. This result proves that ModSecurity works correctly as a Web Application Firewall (WAF). It detected the malicious pattern (AND 1=1) within the HTTP request and blocked it immediately, protecting the application from the attack, as required by the security scenario.

Security Policy for Home Office

To ensure a high level of security in a Home Office environment, the adoption and implementation of the following practices is recommended:

- **Router Hardening: The router's security is strengthened by:**
 - o replacing the default passwords with strong and unique ones,
 - o disabling Remote Management,
 - o disabling the UPnP protocol, which can create uncontrolled port openings.

- **Device and Update Management:** It is necessary to enable automatic security updates for the operating system and the installed applications. At the same time, every work device must have protection software (Antivirus or EDR) as well as an active host-based firewall, so as to reduce the attack surface.
- **Use of a Virtual Private Network (VPN):** Access to corporate resources and services must be carried out exclusively via a VPN, which provides encrypted communication. The use of a VPN ensures both the confidentiality and the integrity of the data transferred, protecting them from interception or traffic-sniffing attacks.
- **Home Network Segmentation:** The home network should be separated into at least two distinct zones, via VLANs or the router's 'Guest Network' function. Devices used for professional purposes should operate on an isolated network, cut off from untrusted or uncontrolled devices (e.g. IoT devices, guest devices or gaming consoles). In this way, the possibility of lateral movement is significantly limited in the event that a device is compromised.
- **Physical Workspace Security:** The physical protection of information is ensured by automatically or manually locking the screen when leaving the workspace, and by applying a 'Clean Desk Policy', so that sensitive documents or information are not accessible to third parties.

Bibliography

- Acunetix. (n.d.). 10 Tips to Secure Your Apache Web Server. <https://www.acunetix.com/blog/articles/10-tips-secure-apache-installation/>
- Apache Software Foundation. (n.d.). Apache HTTP Server Version 2.4 Documentation. <https://httpd.apache.org/docs/2.4/>
- Apache Software Foundation. (n.d.). Security Tips for Apache Server. https://httpd.apache.org/docs/2.4/misc/security_tips.html
- Cloudflare. (n.d.). What is a WAF? | Web Application Firewall explained. <https://www.cloudflare.com/learning/ddos/glossary/web-application-firewall-waf/>
- Computerphile. (2016, May 11). SQL Injection Explained [Video file]. <https://www.youtube.com/watch?v=ciNHn38EyRc>
- DigitalOcean. (2023). Iptables Essentials: Common Firewall Rules and Commands. <https://www.digitalocean.com/community/tutorials/ufw-essentials-common-firewall-rules-and-commands>
- GeeksforGeeks. (2024). iptables command in Linux with Examples. <https://www.geeksforgeeks.org/linux-unix/iptables-command-in-linux-with-examples/>
- Hussein Nasser. (2020, September 20). Apache ModSecurity Tutorial [Video file]. <https://www.youtube.com/watch?v=MB7nQrlP5Yc>
- Linux Manual. (n.d.). Ncat - Netcat for the 21st Century. <https://man7.org/linux/man-pages/man1/ncat.1.html>
- Netfilter Project. (n.d.). iptables(8)-Linux man page. <https://linux.die.net/man/8/iptables>
- NetworkChuck. (2021, July 15). Linux Firewalls (iptables) for Beginners. <https://www.youtube.com/watch?v=v-z6LCcrutw>
- OWASP Foundation. (n.d.). OWASP ModSecurity Core Rule Set (CRS). <https://owasp.org/www-project-modsecurity-core-rule-set/>
- OWASP Foundation. (n.d.). SQL Injection. https://owasp.org/www-community/attacks/SQL_Injection

- Palo Alto Networks. (n.d.). What is a Web Application Firewall (WAF)?.
<https://www.paloaltonetworks.com/cyberpedia/what-is-a-web-application-firewall>
- SpiderLabs. (n.d.). ModSecurity Reference Manual.
<https://github.com/SpiderLabs/ModSecurity/wiki>
- TecMint. (2023). How to Install ModSecurity with Apache on Debian/Ubuntu.
<https://www.tecmint.com/install-modsecurity-apache-debian-ubuntu/>